Dagents: A Functional-Planning Framework for Data Engineering, ML Orchestration, and NL2SQL Applications

Pradyun Devarakonda Dheeraj Rajanala
Programming Languages Course Project
IMT2022525, IMT2022093

Abstract—Dagents is a reusable agentic framework for data engineering, ML automation, workload generation, and local/-global orchestration. The project combines Python services, shared Pydantic contracts, container-ready deployment boundaries, and an OCaml functional planner layer. This final report explains the whole system, with special focus on how functional programming is used to make deterministic planning tasks safer: source validation, extraction planning, schema validation, quality evaluation, transform planning, pipeline DAG validation, model routing, and Kubernetes manifest rendering. The report also covers a new NL2SQL demo application that uses Dagents end-to-end: it takes a database schema and natural-language question, invokes notebook-trained model artifacts through adapters, and shows how core-service, pipeline-service, model-service, LMA, GMA, and the OCaml planner participate in the generation flow. The main result is a polyglot architecture where functional programming does not replace the runtime stack; instead, it provides a typed planner kernel that runtime services can trust and reuse.

Index Terms—functional programming, OCaml, data engineering, NL2SQL, ML orchestration, pipeline planning, Kubernetes, agentic systems

I. Introduction

Modern data and ML applications are rarely a single program. They combine source extraction, quality checks, data transformations, pipeline execution, model training, inference, deployment, and operational monitoring. If these decisions are implemented as scattered conditional logic across multiple services, the system becomes difficult to test and explain.

Dagents addresses this problem by separating two concerns. Runtime services perform side-effecting work: HTTP APIs, source access, model execution, and workflow execution. A functional planner layer performs deterministic translation: structured inputs become validated plans. The project is therefore polyglot by design. Python is used for FastAPI services, ML libraries, and integration. OCaml is used where type structure, pattern matching, and pure functions improve correctness.

This report describes the final state of the project, including the main Dagents stack and a concrete NL2SQL demo application built on top of it. The demo is important because it shows that Dagents is not only a collection of planning modules. It can support an application that accepts schema and natural language input, uses trained model artifacts, and makes the service/planner trace visible to a presenter.

II. Problem Description and Objectives

The core learning objective was to demonstrate where functional programming is useful in an application-scale data engineering system. The intended outcome was not a programming-language compiler. In this project, module names such as dataset_compiler and pipeline_compiler mean domain-specific planners or translators. They lower Dagents specifications into runtime plans. They do not parse or compile a general programming language.

The project objectives were:

- build a reusable ML/data-engineering framework with local and aggregate agent boundaries;
- implement typed functional planner kernels in OCaml;
- expose planner functionality to services through a process boundary;
- demonstrate full-app usage through service health, topology, pipeline, model, workload, LMA, and GMA flows;
- add an NL2SQL demo app that uses Dagents rather than a separate generic backend;
- provide a presentation-ready demo package, documentation, and tests.

III. System Overview

Dagents is organized around reusable services and shared contracts. The major runtime components are:

- LMA: Local Monitoring Agent for source-scoped execution.
- GMA: Global Monitoring Agent for fleet-wide coordination and aggregate data processing.
- core-service: service catalog, topology, and workload-/manifest generation.
- pipeline-service: JSON workflow registration and execution.
- model-service: reusable ML dataset catalog, training, and model-job support.
- bindings/ocaml: pure functional planner modules.
- nl2sql-demo: a React and FastAPI demo app that proves Dagents can host a downstream application.

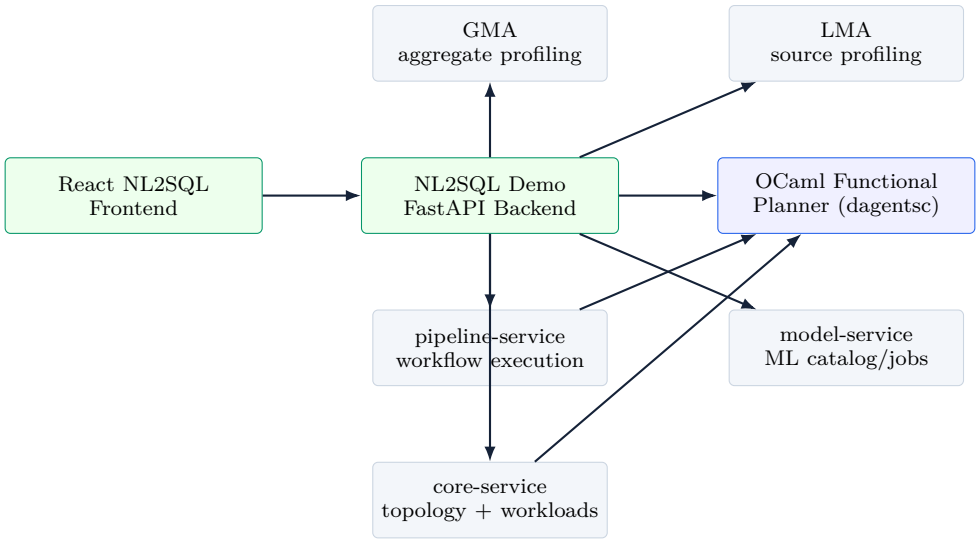

Fig. 1. Dagents application architecture. Runtime services perform side effects; the OCaml layer performs deterministic functional planning.

Figure 1 shows the main structure. The NL2SQL app uses the same services as the rest of Dagents. It does not implement a standalone generic backend. The backend imports shared Dagents domain models, calls the OCaml planner through dagents_runner, and touches core-service, pipeline-service, model-service, LMA, and GMA in its trace.

IV. Functional Planner Layer

The OCaml workspace is located under bindings/ocaml. The modules are shown below:

```
bindings/ocaml
|-- lib/common_ir
|-- lib/dataset_compiler
|-- lib/pipeline_compiler
|-- lib/model_router
|-- lib/manifest_compiler
`-- bin/dagentsc.ml
```

A. Common IR

common_ir defines the typed domain representation shared by the planner modules. It models source kinds, records, schema fields, quality operators, severities, transform operations, pipeline steps, model families, task types, partition strategies, and workload kinds. Algebraic data types are important here because they make the planner domain explicit. For example, a source is not just a dictionary; it is one of a closed set of valid source forms such as inline, Postgres, MongoDB, or object storage.

B. Dataset Planner

dataset_compiler is a source and dataset planner. It validates source specifications, constructs extraction plans, infers schemas, validates record shape against contracts, evaluates quality rules, and plans transformations. Quality checks are severity-aware: warning-level failures are reported, while error-level failures contribute to an aggregate blocking flag. Partitioning is also planned here, including time-window partitioning based on source options such as timeField and timeWindow.

C. Pipeline Planner

pipeline_compiler validates JSON workflow DAGs. It rejects duplicate step IDs, unknown dependencies, and cycles. When a pipeline is valid, it returns a topological execution order and maps steps to execution targets. This is a natural functional programming problem because the planner is a pure graph transformation with well-defined failure modes.

D. Model Router

model_router maps a dataset profile and task type to candidate model families. For example, forecasting can route toward recurrent families such as GRU/LSTM, while anomaly detection can route toward autoencoder-style models. The router does not train models. It decides what kind of model execution should be requested by runtime services.

E. Manifest Planner

manifest_compiler translates workload specifications into Kubernetes YAML for Deployments, Jobs, CronJobs, Services, ConfigMaps, and ServiceAccounts. This replaces ad hoc YAML string construction with a typed rendering path.

F. Process Boundary

The CLI dagentsc is the integration boundary. Python services send JSON payloads to the OCaml planner and read JSON results back. This was chosen over direct FFI because subprocess integration is simple to test, easier to isolate, and portable across Python and Spring services.

V. Dagents Runtime Services

A. LMA and GMA

The LMA is responsible for source-scoped work: profiling local datasets, executing local model runs, and tracking bundle/run state. The GMA is responsible for aggregate coordination: registering agents, tracking deployments, profiling assimilated datasets, and running aggregate model jobs. This split mirrors realistic deployments where some work happens near a tenant, stream, or application boundary, while other work happens centrally.

B. Core Service

Core service exposes framework metadata and workload planning: /api/v1/health, /api/v1/services, /api/v1/topology, /api/v1/manifests/pods, and /api/v1/workloads:compile. In the NL2SQL demo, the backend calls core service to retrieve topology and compile a demo workload plan for the NL2SQL backend.

C. Pipeline Service

Pipeline service registers and runs JSON workflows. Built-in steps include context enrichment, filtering, summarization, field projection, dataset profiling, and model-job execution. The service can call the OCaml planner for pipeline validation. In NL2SQL, the backend registers a schema-profiling pipeline, runs it against schema-record payloads, and includes the result in the Dagents trace.

D. Model Service

Model service owns reusable ML infrastructure: dataset catalog, train endpoint, model jobs, classification/regression/forecasting checks, and source registration. NL2SQL uses the service for model catalog/job visibility while NL2SQL-specific adapters load the notebook-generated text-to-SQL artifacts.

VI. NL2SQL Demo Application

The NL2SQL application demonstrates how a new app can be built on Dagents. It accepts a natural-language question and a table schema, renders the schema as DDL, formats the prompt according to the selected model family, runs the model adapter, and returns SQL plus a detailed Dagents trace.

A. Model Artifacts

The notebooks under notebooks/ produced model artifacts under models/. The implemented adapters support:

- CodeQwen LoRA adapters using a Qwen chat template with a SQL-only system instruction and a Context/Question user message;
- CodeT5/T5-style sequence-to-sequence artifacts using question: {question} context: {DDL}.

The backend lazily extracts zip artifacts into a cache directory and loads the selected model when first requested. The deterministic fallback remains for demo reliability, but the default model is codet5p_spider_model when the runtime dependencies are available.

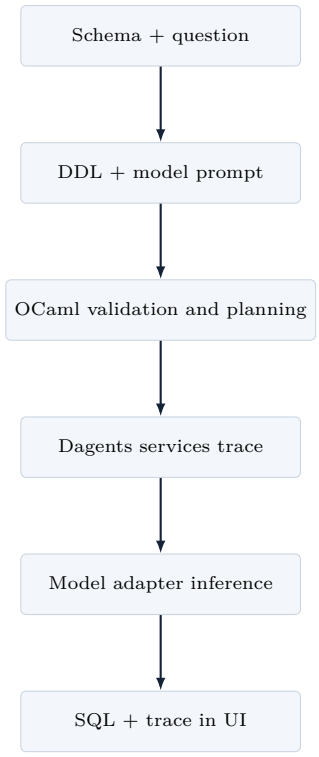

Fig. 2. NL2SQL flow through the Dagents framework.

B. Request Flow

The backend builds a trace with the following steps:

1) validate an inline schema source with the OCaml SourceSpec planner;
2) compile an extraction plan;
3) validate schema records against a schema contract;
4) evaluate quality rules for table name, column name, and type presence;
5) compile a pipeline DAG;
6) route a model task through the OCaml model router;
7) call core-service for catalog/topology and workload planning;
8) call pipeline-service to register and run a schema workflow;
9) call model-service for catalog and recent job visibility;
10) call LMA and GMA dataset profile endpoints.

This is what makes the demo a Dagents app rather than a standalone NL2SQL wrapper.

VII. How the Project Can Be Used

Dagents can be used in several ways beyond the course demo:

- Data quality gateway: validate sources, schemas, and quality rules before expensive runtime jobs start.
- Reusable ML orchestration layer: run source-level and aggregate models through a shared LMA/GMA control plane.

TABLE I
Functional and runtime responsibilities

Concern	OCaml planner	Runtime services
Source handling	validate shape, plan extraction, partitioning	connect to data sources, fetch records
Schema and quality	infer/validate schemas, compute blocking status	persist reports, expose APIs
Pipelines	check DAGs, order steps	execute handlers and store runs
Models	route task to candidate model family	train, load, infer, persist artifacts
Workloads	render deterministic YAML	publish topology, compile plans, deploy externally

- Workflow planning: register JSON pipelines while using a typed planner to reject cycles and invalid dependencies.
- Deployment generation: render Kubernetes workload manifests from typed workload specs.
- NL2SQL and analyst tools: build applications that turn natural language into data actions while exposing validation, routing, and service traceability.
- Teaching functional programming: show how algebraic data types, pure functions, and pattern matching can improve a real service architecture.

VIII. Implementation Highlights

Table I summarizes the core design principle. OCaml owns deterministic policy and translation. Python owns side effects and ecosystem integration. This split also improves testing. Pure planner functions can be tested without databases, Docker, GPUs, or external APIs.

The NL2SQL backend imports shared Dagents models and wrappers:

```
from agents.common.infrastructure.dagents_runner import (
    compile_dataset_extraction,
    evaluate_dataset_quality,
    run_dagentsc,
    validate_dataset_schema,
    validate_dataset_source,
)
```

This is the key integration point. It gives application code access to the functional layer without embedding OCaml in-process.

IX. Testing and Demo Evidence

The implemented test plan covers both the functional kernels and service integration wrappers:

```
opam exec -- dune test
opam exec -- dune build ./bin/dagentsc.exe
.venv/bin/python -m pytest \
  agents/tests/test_dagents_runner.py \
  services/nl2sql-demo/tests/test_nl2sql_api.py
npm run build
```

OCaml tests cover source validation, invalid source rejection, time-window partition planning, schema contract validation, quality reporting, transform planning/application, duplicate pipeline steps, unknown dependencies, cycle rejection, model routing, manifest rendering, and JSON codec round trips.

The NL2SQL application was also run locally with the Dagents services active. The backend reported every service reachable and a full generation trace returned ok for planner steps, core-service, pipeline-service, model-service, LMA, GMA, and service status. A real CodeT5 model artifact was loaded through the backend engine and returned used_fallback: false.

X. Design Decisions and Tradeoffs

A. Why OCaml?

OCaml is used because the planner logic benefits from algebraic data types, exhaustive pattern matching, and immutable transformations. These features are especially helpful for cases where the system must know every valid source kind, quality operator, workload kind, or pipeline step category.

B. Why Not Python Only?

Python is excellent for APIs, ML, and integration, but the planner layer is a place where static structure reduces ambiguity. The project does not argue that all services should be rewritten in OCaml. It argues that the deterministic planning kernel is a good target for functional programming.

C. Why JSON Subprocess Instead of FFI?

The subprocess boundary is less elegant than direct function calls but easier to operate. It isolates failures, works across languages, avoids ABI concerns, and makes input/output contracts visible in demo scripts and service tests.

D. Limitations

The OCaml layer intentionally avoids I/O. It does not connect to databases, train models, call Kubernetes, or access object storage. This keeps the planner pure, but it means runtime adapters still need production hardening. NL2SQL model quality also depends on the supplied artifacts, prompt format, model size, and available hardware.

XI. Work Distribution

The project was divided into two primary roles:

- Pradyun Devarakonda: OCaml functional planner design, dagentsc integration, dataset/pipeline/model/-manifest planning, demo scripts, report integration, and NL2SQL backend orchestration.
- Dheeraj Rajanala: Dagents service architecture support, LMA/GMA and service-flow validation, NL2SQL model artifact exploration, frontend/demo usability, and presentation/testing support.

Both members contributed to integration, testing, and final presentation material. The work distribution reflects the course goal: connect functional programming ideas to a working application rather than presenting isolated examples.

XII. Conclusion

Dagents demonstrates a practical way to use functional programming inside a larger data and ML framework. The OCaml layer supplies typed, deterministic planning. Python services supply APIs, runtime execution, and ML ecosystem support. The NL2SQL demo shows that new applications can be built on top of Dagents by reusing service contracts, functional planners, and model adapters.

The most important lesson is architectural: functional programming does not need to own the whole application to be valuable. It can be introduced as a small, high-leverage kernel where correctness depends on structured data, explicit cases, and deterministic transformations.

References

[1] R. Milner, "A theory of type polymorphism in programming," Journal of Computer and System Sciences, vol. 17, no. 3, pp. 348–375, 1978.

[2] X. Leroy, D. Doligez, A. Frisch, J. Garrigue, D. Remy, and J. Vouillon, The OCaml System: Documentation and User's Manual. OCaml.org. [Online]. Available: https://ocaml.org/manual/

[3] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in Proc. 6th USENIX Symposium on Operating Systems Design and Implementation, 2004, pp. 137–150.

[4] C. Raffel et al., "Exploring the limits of transfer learning with a unified text-to-text transformer," Journal of Machine Learning Research, vol. 21, no. 140, pp. 1–67, 2020.

[5] Y. Wang, W. Wang, S. R. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in Proc. EMNLP, 2021.

[6] T. Yu et al., "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in Proc. EMNLP, 2018.

[7] E. Hu et al., "LoRA: Low-rank adaptation of large language models," in Proc. ICLR, 2022.

[8] The Kubernetes Authors, "Workloads," Kubernetes Documentation. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/